

Take-Home Assignment

AI & Innovation Engineer — Time Internet

Context

Time Internet is a broadband provider. Every month a batch of customers' contracts approach expiry, and retention agents need to work the list quickly and reach out with a personalised recontract pitch. We want an internal tool that helps them do this, with an AI layer that's genuinely reliable rather than a demo. This role is full-stack with real AI ownership, so the assignment spans frontend, backend, and the AI integration end to end.

The Task

Build a web application where a retention agent can:

- Browse, search, filter, and sort customers whose contracts are expiring, with pagination
- View a customer's details — plan, tenure, and usage history
- Generate a personalised recontract pitch using an LLM that references the customer's actual data (plan, tenure, usage) so it reads tailored, not generic
- Generate pitches in bulk for a filtered segment, with progress tracking
- See, copy, and regenerate pitches

Fake the customer data (seed 50–100 records). Focus on end-to-end structure and a reliable AI layer over visual polish.

Frontend Requirements

The agent-facing UI should include:

- A dashboard summary of customers expiring this month, broken down by plan tier
- Filter and sort the customer list by plan, tenure, or usage
- A detail view showing customer info and the generated pitch side by side
- Visual indicators for pitch status (not generated / generating / ready / failed)
- Stream the LLM response so the agent sees the pitch generate in real time

No design system required — we're evaluating component structure, state management, and data flow.

AI & Production Considerations

This is where we most want to see your thinking, given the role. Address as many as you can in code or in the README, with your tradeoffs:

- Grounding — how you structure the prompt/context so the pitch reliably reflects the customer's real data and doesn't hallucinate plans or numbers
- Caching / idempotency — avoid regenerating a pitch when nothing relevant changed for a customer; explain your cache key design
- Fallback — if the LLM provider is unavailable, the system should degrade gracefully (fallback model, cached response, or clear error state)
- Rate limiting & concurrency — bulk generation shouldn't overwhelm the LLM API; show how you handle batching and backpressure
- Failure handling — when a pitch fails mid-batch, the agent should see which succeeded and which didn't
- Cost & observability — how you'd track token cost and monitor the AI layer in production

Tech Constraints

- TypeScript (not JavaScript) for the frontend
- React or Next.js for the frontend
- Python for the backend / AI integration layer
- Any LLM API (OpenAI, Gemini, Claude, open-source — your call)
- Any database or data layer you see fit
- Containerize with Docker (docker-compose for local dev)
- Deploy somewhere other than Vercel or Netlify, or document your deployment approach and architecture in the README

What We're Evaluating

- End-to-end ownership — how you architect across frontend, backend, and the AI layer
- AI reliability thinking — grounding, fallback, cost, and failure modes, not just a working prompt
- Architecture decisions and tradeoffs — your README matters as much as the code
- Clean, readable, well-tested code
- Git history — commit frequently so we can see how you work

You're welcome to use any tools including AI assistants. Be prepared to walk through your code and decisions during a follow-up technical round.

Logistics

Time box: 5 days from receiving this brief. We expect 6–8 hours of actual work.

Submission: GitHub repo link + deployed URL (if applicable) + README with your architecture and design notes.

Follow-up: A technical round where we walk through your code and decisions together.